

Diabetes Prediction using Machine Learning and Deep Learning

CS634 Data Mining - Final Project

Name: Veekshitha Beladara Nanjegowda

Date: November 2025

This notebook implements three binary classification algorithms to predict diabetes:

1. **Random Forest** (Mandatory)
2. **LSTM** (Deep Learning)
3. **SVM** (Traditional ML)

We use **10-fold cross-validation** and calculate all metrics manually.

1. Import Libraries

```
In [27]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

from sklearn.model_selection import KFold, train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, roc_curve, auc, brier_score_loss

from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

sns.set_style("whitegrid")
plt.rcParams['figure.facecolor'] = 'white'
plt.rcParams['axes.facecolor'] = 'white'

print("All libraries imported successfully!")
```

All libraries imported successfully!

2. Load and Explore Data

```
In [28]: data = pd.read_csv('diabetes.csv')

print("First 5 rows of data:")
print(data.head())

print("\nDataset Information:")
print(data.info())

# Statistical summary
print("\nStatistical Summary:")
print(data.describe())
```

First 5 rows of data:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	\
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 768 entries, 0 to 767

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	Pregnancies	768 non-null	int64
1	Glucose	768 non-null	int64
2	BloodPressure	768 non-null	int64
3	SkinThickness	768 non-null	int64
4	Insulin	768 non-null	int64
5	BMI	768 non-null	float64
6	DiabetesPedigreeFunction	768 non-null	float64
7	Age	768 non-null	int64
8	Outcome	768 non-null	int64

dtypes: float64(2), int64(7)

memory usage: 54.1 KB

None

Statistical Summary:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	\
count	768.000000	768.000000	768.000000	768.000000	768.000000	
mean	3.845052	120.894531	69.105469	20.536458	79.799479	
std	3.369578	31.972618	19.355807	15.952218	115.244002	
min	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	1.000000	99.000000	62.000000	0.000000	0.000000	
50%	3.000000	117.000000	72.000000	23.000000	30.500000	
75%	6.000000	140.250000	80.000000	32.000000	127.250000	
max	17.000000	199.000000	122.000000	99.000000	846.000000	

	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000
mean	31.992578	0.471876	33.240885	0.348958
std	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.078000	21.000000	0.000000
25%	27.300000	0.243750	24.000000	0.000000
50%	32.000000	0.372500	29.000000	0.000000
75%	36.600000	0.626250	41.000000	1.000000
max	67.100000	2.420000	81.000000	1.000000

3. Data Preprocessing

```
In [29]: columns_to_fix = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in columns_to_fix:
    data[col] = data[col].replace(0, np.nan)
    data[col] = data[col].fillna(data[col].median())

print("Missing values handled successfully!")
print("\nChecking for any remaining missing values:")
print(data.isnull().sum())
```

Missing values handled successfully!

Checking for any remaining missing values:

Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

4. Beautiful Visualizations for Report



Graph 1: Class Distribution (Pie Chart)

```
In [30]: plt.figure(figsize=(10, 7))

counts = data['Outcome'].value_counts()
colors = ['#3498db', '#e74c3c']
explode = (0.05, 0.05)

plt.pie(counts, labels=['No Diabetes', 'Has Diabetes'],
        autopct='%1.1f%%', startangle=90, colors=colors,
        explode=explode, shadow=True,
        textprops={'fontsize': 14, 'weight': 'bold'})

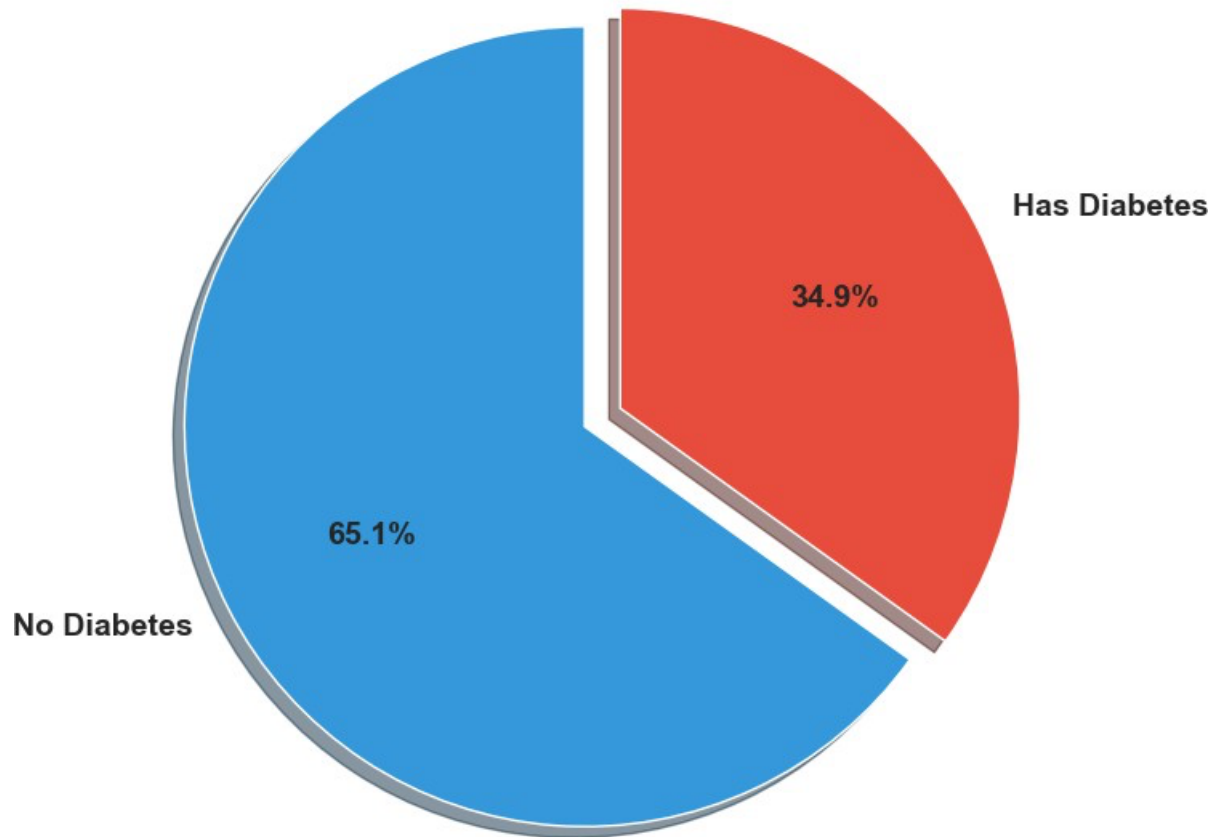
plt.title('Diabetes Distribution in Dataset',
        fontsize=18, weight='bold', pad=20)
plt.tight_layout()
plt.savefig('graph1_class_distribution.png', dpi=300, bbox_inches='tight')
plt.show()

no_diabetes = data[data['Outcome']==0].shape[0]
has_diabetes = data[data['Outcome']==1].shape[0]
total = data.shape[0]

print(f"\nClass Distribution:")
```

```
print(f"No Diabetes: {no_diabetes} ({no_diabetes/total*100:.1f}%)")
print(f"Has Diabetes: {has_diabetes} ({has_diabetes/total*100:.1f}%)")
print("\n✔ Graph 1 saved as 'graph1_class_distribution.png'")
```

Diabetes Distribution in Dataset



 **Class Distribution:**
 No Diabetes: 500 (65.1%)
 Has Diabetes: 268 (34.9%)

✔ Graph 1 saved as 'graph1_class_distribution.png'

Graph 2: Age Distribution by Diabetes Status

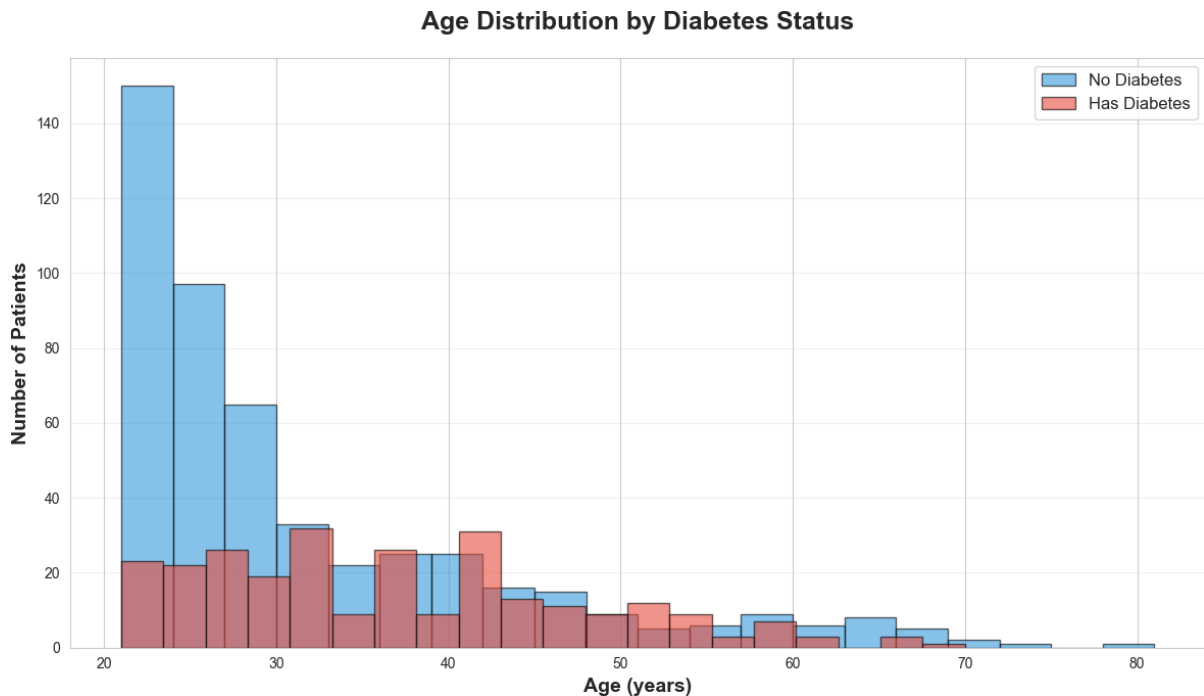
```
In [31]: # Age distribution histogram
plt.figure(figsize=(12, 7))

no_diabetes = data[data['Outcome'] == 0]['Age']
has_diabetes = data[data['Outcome'] == 1]['Age']

plt.hist(no_diabetes, bins=20, alpha=0.6, label='No Diabetes',
         color='#3498db', edgecolor='black')
plt.hist(has_diabetes, bins=20, alpha=0.6, label='Has Diabetes',
         color='#e74c3c', edgecolor='black')
```

```
plt.xlabel('Age (years)', fontsize=14, weight='bold')
plt.ylabel('Number of Patients', fontsize=14, weight='bold')
plt.title('Age Distribution by Diabetes Status',
          fontsize=18, weight='bold', pad=20)
plt.legend(fontsize=12, loc='upper right')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('graph2_age_distribution.png', dpi=300, bbox_inches='tight')
plt.show()

print("Graph 2 saved as 'graph2_age_distribution.png'")
```



Graph 2 saved as 'graph2_age_distribution.png'



Graph 3: Key Features Comparison (Box Plots)

```
In [32]: # Multiple box plots for feature comparison
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
fig.suptitle('Key Medical Features by Diabetes Status',
             fontsize=20, weight='bold', y=1.02)

features_to_plot = ['Glucose', 'BMI', 'Age', 'BloodPressure', 'Insulin', 'Pregnancy']
colors_box = ['#3498db', '#e74c3c']

for idx, feature in enumerate(features_to_plot):
    row = idx // 3
    col = idx % 3
    ax = axes[row, col]

    no_diab = data[data['Outcome'] == 0][feature]
    has_diab = data[data['Outcome'] == 1][feature]

    box_data = [no_diab, has_diab]
    bp = ax.boxplot(box_data, labels=['No Diabetes', 'Has Diabetes'],
```

```

        patch_artist=True, widths=0.6)

    for patch, color in zip(bp['boxes'], colors_box):
        patch.set_facecolor(color)
        patch.set_alpha(0.7)

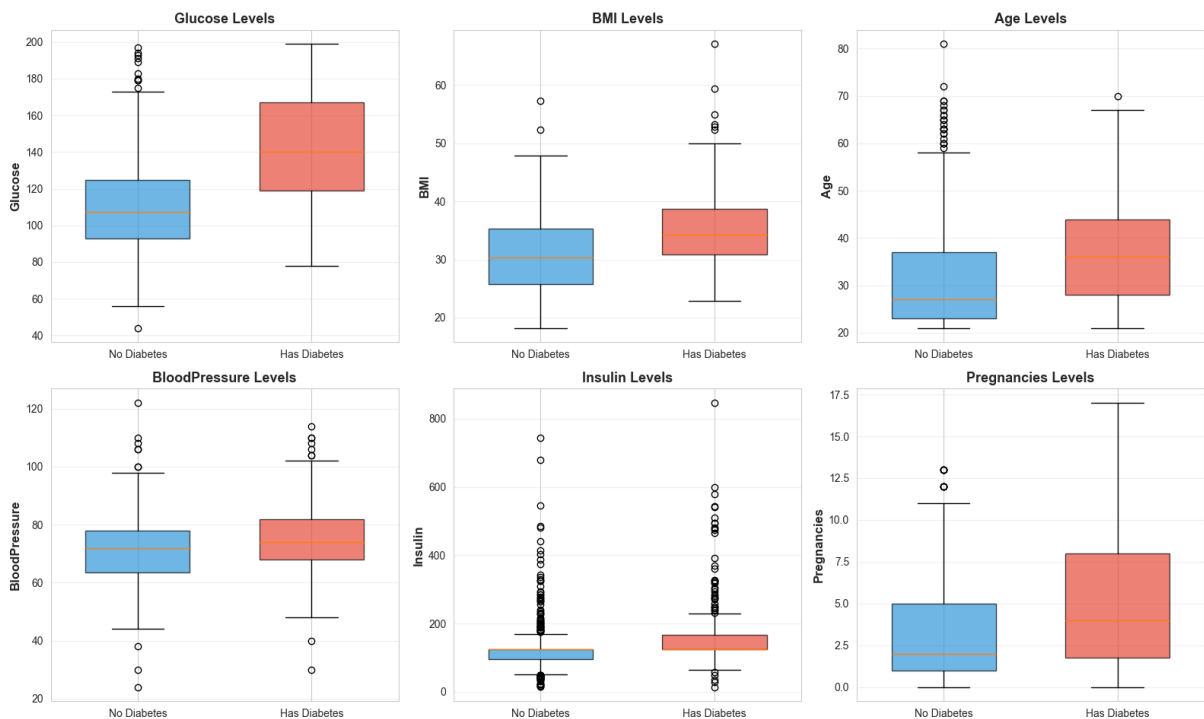
    ax.set_ylabel(feature, fontsize=12, weight='bold')
    ax.set_title(f'{feature} Levels', fontsize=13, weight='bold')
    ax.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.savefig('graph3_feature_comparison.png', dpi=300, bbox_inches='tight')
plt.show()

print("Graph 3 saved as 'graph3_feature_comparison.png'")

```

Key Medical Features by Diabetes Status



Graph 3 saved as 'graph3_feature_comparison.png'



Graph 4: Glucose vs BMI Scatter Plot

```

In [33]: plt.figure(figsize=(12, 8))

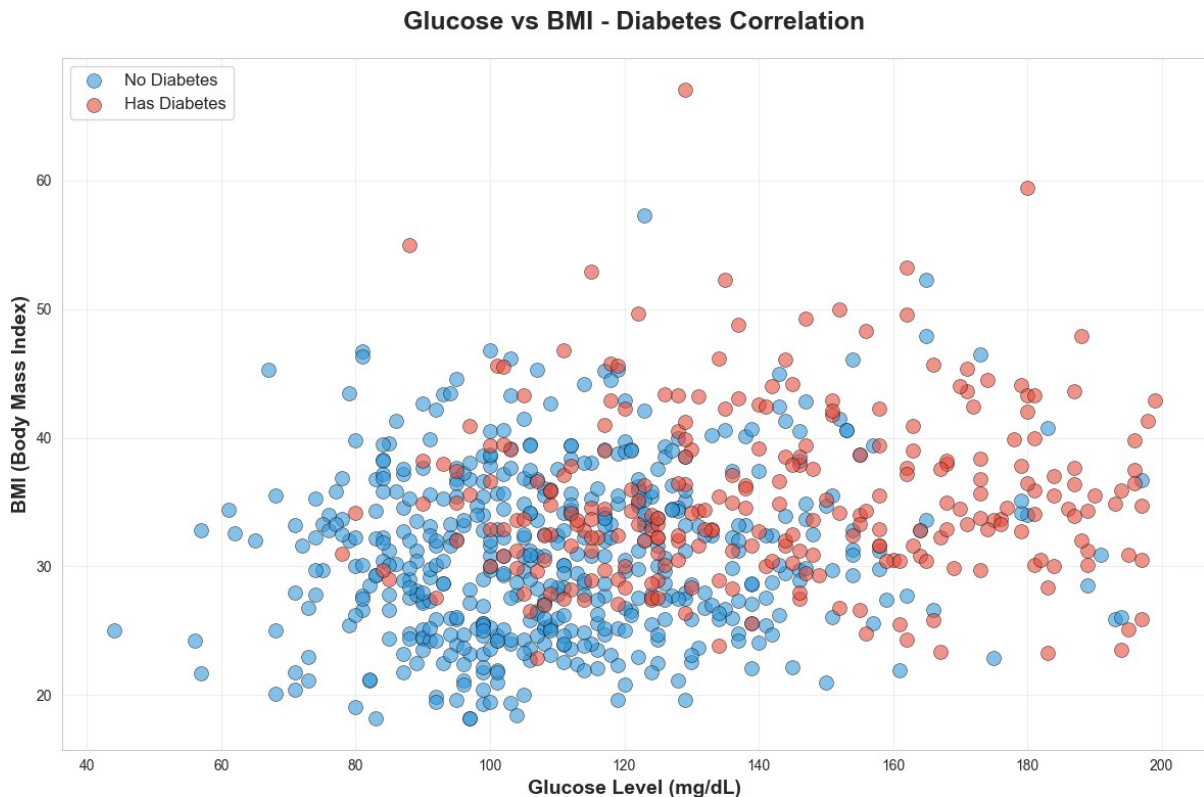
no_diab = data[data['Outcome'] == 0]
has_diab = data[data['Outcome'] == 1]

plt.scatter(no_diab['Glucose'], no_diab['BMI'],
            alpha=0.6, s=100, c='#3498db', label='No Diabetes',
            edgecolors='black', linewidth=0.5)
plt.scatter(has_diab['Glucose'], has_diab['BMI'],
            alpha=0.6, s=100, c='#e74c3c', label='Has Diabetes',
            edgecolors='black', linewidth=0.5)

```

```
plt.xlabel('Glucose Level (mg/dL)', fontsize=14, weight='bold')
plt.ylabel('BMI (Body Mass Index)', fontsize=14, weight='bold')
plt.title('Glucose vs BMI - Diabetes Correlation',
          fontsize=18, weight='bold', pad=20)
plt.legend(fontsize=12, loc='upper left')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('graph4_glucose_bmi_scatter.png', dpi=300, bbox_inches='tight')
plt.show()

print("Graph 4 saved as 'graph4_glucose_bmi_scatter.png'")
```



Graph 4 saved as 'graph4_glucose_bmi_scatter.png'



Graph 5: Correlation Heatmap (Clean Version)

```
In [34]: # Beautiful correlation heatmap
plt.figure(figsize=(12, 10))
correlation = data.corr()

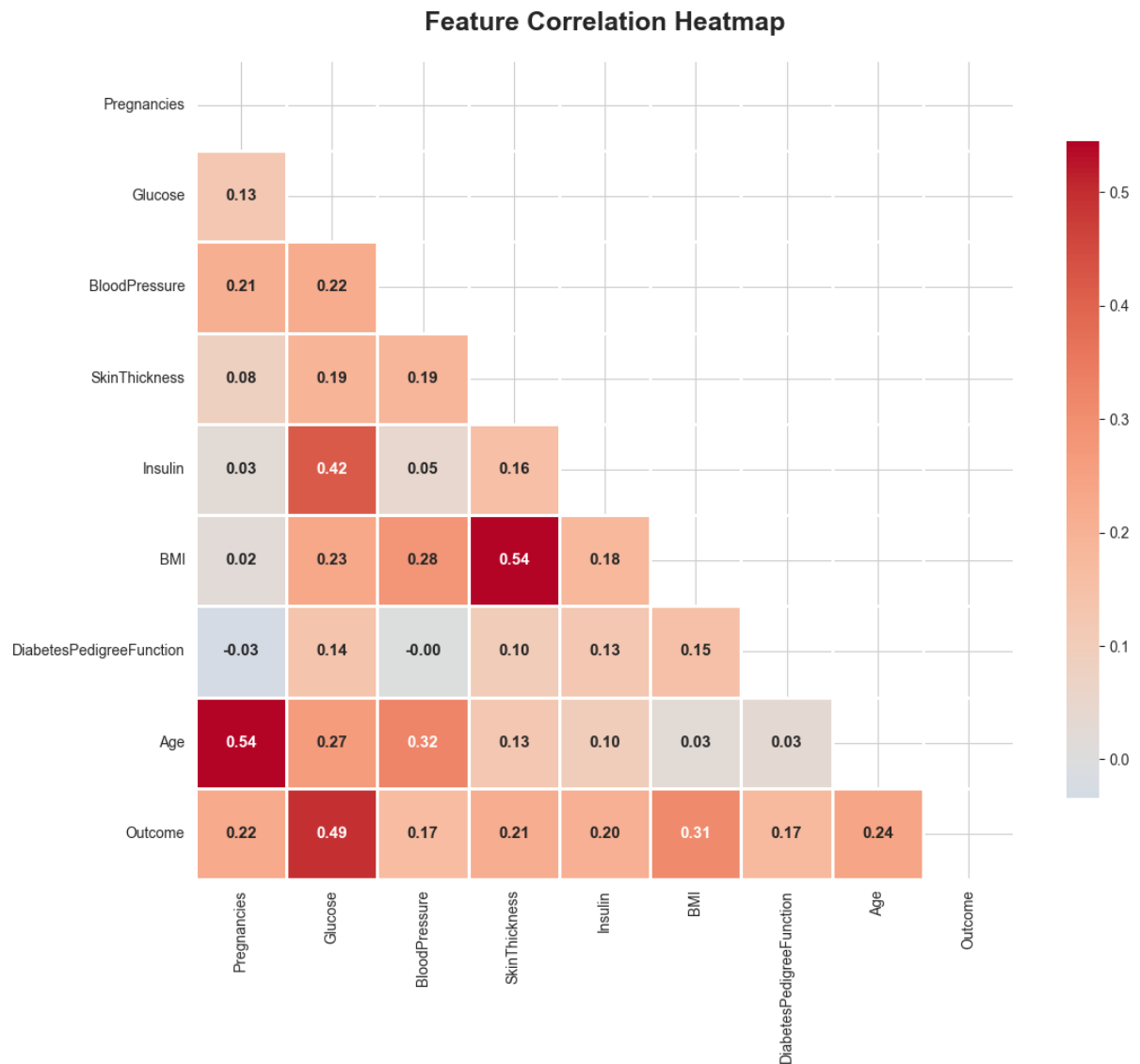
mask = np.triu(np.ones_like(correlation, dtype=bool))

sns.heatmap(correlation, mask=mask, annot=True, fmt='.2f',
            cmap='coolwarm', center=0, square=True,
            linewidths=1, linecolor='white',
            cbar_kws={'shrink': 0.8},
            annot_kws={'size': 11, 'weight': 'bold'})

plt.title('Feature Correlation Heatmap', fontsize=18, weight='bold', pad=20)
plt.tight_layout()
plt.savefig('graph5_correlation_heatmap.png', dpi=300, bbox_inches='tight')
```

```
plt.show()

print("Graph 5 saved as 'graph5_correlation_heatmap.png'")
```



Graph 5 saved as 'graph5_correlation_heatmap.png'

5. Prepare Data for Modeling

```
In [35]: x = data.drop('Outcome', axis=1)
y = data['Outcome']

print(f"Features shape: {x.shape}")
print(f"Target shape: {y.shape}")
print(f"\nFeature columns: {list(x.columns)}")
```

Features shape: (768, 8)

Target shape: (768,)

Feature columns: ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age']

6. Define Metrics Calculation Functions

We calculate all metrics **manually** (except confusion matrix generation)

```
In [36]: def calculate_all_metrics(actual, predicted, predicted_proba=None):

    cm = confusion_matrix(actual, predicted, labels=[1, 0])
    tp = cm[0][0] # True Positive
    fn = cm[0][1] # False Negative
    fp = cm[1][0] # False Positive
    tn = cm[1][1] # True Negative

    tpr = tp / (tp + fn) # True Positive Rate (Sensitivity/Recall)
    tnr = tn / (tn + fp) # True Negative Rate (Specificity)
    fpr = fp / (tn + fp) # False Positive Rate
    fnr = fn / (tp + fn) # False Negative Rate

    precision = tp / (tp + fp) if (tp + fp) > 0 else 0
    recall = tpr
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0
    accuracy = (tp + tn) / (tp + tn + fp + fn)
    error_rate = (fp + fn) / (tp + tn + fp + fn)

    balanced_acc = (tpr + tnr) / 2

    # TSS (True Skill Statistic)
    tss = tpr - fpr

    # HSS (Heidke Skill Score)
    numerator = 2 * (tp * tn - fp * fn)
    denominator = (tp + fn) * (fn + tn) + (tp + fp) * (fp + tn)
    hss = numerator / denominator if denominator > 0 else 0

    # Calculate P and N
    p = tp + fn # Total actual positives
    n = tn + fp # Total actual negatives

    metrics = {
        'TP': tp,
        'TN': tn,
        'FP': fp,
        'FN': fn,
        'P': p,
        'N': n,
        'TPR': tpr,
        'TNR': tnr,
        'FPR': fpr,
        'FNR': fnr,
        'Precision': precision,
        'Recall': recall,
        'F1': f1,
        'Accuracy': accuracy,
        'Error_Rate': error_rate,
        'Balanced_Acc': balanced_acc,
```

```

        'TSS': tss,
        'HSS': hss
    }

    if predicted_proba is not None:
        fpr_curve, tpr_curve, _ = roc_curve(actual, predicted_proba)
        roc_auc = auc(fpr_curve, tpr_curve)
        bs = brier_score_loss(actual, predicted_proba)

        climatology = np.mean(actual)
        bs_ref = np.mean((actual - climatology) ** 2)
        bss = 1 - (bs / bs_ref) if bs_ref > 0 else 0

        metrics['AUC'] = roc_auc
        metrics['Brier_Score'] = bs
        metrics['BSS'] = bss

    return metrics

print("Metrics calculation function defined successfully!")

```

Metrics calculation function defined successfully!

7. Set Up 10-Fold Cross-Validation

```

In [37]: num_folds = 10
kfold = KFold(n_splits=num_folds, shuffle=True, random_state=42)

rf_results = []
lstm_results = []
svm_results = []

print(f"10-Fold Cross-Validation initialized with {num_folds} folds")

```

10-Fold Cross-Validation initialized with 10 folds

8. Train and Evaluate Models with 10-Fold CV

We'll train all three models on each fold and calculate metrics.

Note: This cell will take 3-5 minutes to run. Be patient! 🕒

```

In [38]: fold_num = 0

print("Starting 10-Fold Cross-Validation...\n")

for train_idx, test_idx in kfold.split(x):
    fold_num += 1
    print(f"{' '*60}")
    print(f"Processing Fold {fold_num}/{num_folds}")
    print(f"{' '*60}")

```

```

# Split data for this fold
x_train = x.iloc[train_idx]
x_test = x.iloc[test_idx]
y_train = y.iloc[train_idx]
y_test = y.iloc[test_idx]

scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)

print(f"Train size: {len(x_train)}, Test size: {len(x_test)}")

# =====
# 1. RANDOM FOREST
# =====
print("\n[1/3] Training Random Forest...")
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(x_train_scaled, y_train)

# Predictions
rf_pred = rf_model.predict(x_test_scaled)
rf_pred_proba = rf_model.predict_proba(x_test_scaled)[: , 1]

# Calculate metrics
rf_metrics = calculate_all_metrics(y_test, rf_pred, rf_pred_proba)
rf_metrics['Fold'] = fold_num
rf_results.append(rf_metrics)
print(f"Random Forest Accuracy: {rf_metrics['Accuracy']:.4f}")

# =====
# 2. SVM
# =====
print("\n[2/3] Training SVM...")
svm_model = SVC(kernel='rbf', probability=True, random_state=42)
svm_model.fit(x_train_scaled, y_train)

# Predictions
svm_pred = svm_model.predict(x_test_scaled)
svm_pred_proba = svm_model.predict_proba(x_test_scaled)[: , 1]

# Calculate metrics
svm_metrics = calculate_all_metrics(y_test, svm_pred, svm_pred_proba)
svm_metrics['Fold'] = fold_num
svm_results.append(svm_metrics)
print(f"SVM Accuracy: {svm_metrics['Accuracy']:.4f}")

# =====
# 3. LSTM
# =====
print("\n[3/3] Training LSTM...")

x_train_lstm = x_train_scaled.reshape((x_train_scaled.shape[0], x_train_scaled.
x_test_lstm = x_test_scaled.reshape((x_test_scaled.shape[0], x_test_scaled.shap

# Build LSTM model
lstm_model = Sequential([

```

```

        LSTM(64, activation='relu', input_shape=(x_train_lstm.shape[1], 1)),
        Dropout(0.2),
        Dense(32, activation='relu'),
        Dense(1, activation='sigmoid')
    ])

    lstm_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accu

    lstm_model.fit(x_train_lstm, y_train, epochs=50, batch_size=32, verbose=0)

    # Predictions
    lstm_pred_proba = lstm_model.predict(x_test_lstm, verbose=0).flatten()
    lstm_pred = (lstm_pred_proba > 0.5).astype(int)

    # Calculate metrics
    lstm_metrics = calculate_all_metrics(y_test, lstm_pred, lstm_pred_proba)
    lstm_metrics['Fold'] = fold_num
    lstm_results.append(lstm_metrics)
    print(f"LSTM Accuracy: {lstm_metrics['Accuracy']:.4f}")
    print()

print(f"\n{'='*60}")
print("All folds completed successfully!")
print(f"{'='*60}")

```

Starting 10-Fold Cross-Validation...

=====
Processing Fold 1/10
=====

Train size: 691, Test size: 77

[1/3] Training Random Forest...

Random Forest Accuracy: 0.6883

[2/3] Training SVM...

SVM Accuracy: 0.7403

[3/3] Training LSTM...

LSTM Accuracy: 0.6883

=====
Processing Fold 2/10
=====

Train size: 691, Test size: 77

[1/3] Training Random Forest...

Random Forest Accuracy: 0.7922

[2/3] Training SVM...

SVM Accuracy: 0.8052

[3/3] Training LSTM...

LSTM Accuracy: 0.7662

=====
Processing Fold 3/10
=====

Train size: 691, Test size: 77

[1/3] Training Random Forest...

Random Forest Accuracy: 0.7403

[2/3] Training SVM...

SVM Accuracy: 0.7403

[3/3] Training LSTM...

LSTM Accuracy: 0.7273

=====
Processing Fold 4/10
=====

Train size: 691, Test size: 77

[1/3] Training Random Forest...

Random Forest Accuracy: 0.8182

[2/3] Training SVM...

SVM Accuracy: 0.8442

[3/3] Training LSTM...

LSTM Accuracy: 0.7922

=====
Processing Fold 5/10
=====

Train size: 691, Test size: 77

[1/3] Training Random Forest...

Random Forest Accuracy: 0.8182

[2/3] Training SVM...

SVM Accuracy: 0.7922

[3/3] Training LSTM...

LSTM Accuracy: 0.8052

=====
Processing Fold 6/10
=====

Train size: 691, Test size: 77

[1/3] Training Random Forest...

Random Forest Accuracy: 0.6883

[2/3] Training SVM...

SVM Accuracy: 0.7143

[3/3] Training LSTM...

LSTM Accuracy: 0.6753

=====
Processing Fold 7/10
=====

Train size: 691, Test size: 77

[1/3] Training Random Forest...

Random Forest Accuracy: 0.8052

[2/3] Training SVM...

SVM Accuracy: 0.7922

[3/3] Training LSTM...

LSTM Accuracy: 0.8182

=====
Processing Fold 8/10
=====

Train size: 691, Test size: 77

[1/3] Training Random Forest...

Random Forest Accuracy: 0.7273

[2/3] Training SVM...

SVM Accuracy: 0.7403

[3/3] Training LSTM...

LSTM Accuracy: 0.7532

```
=====
Processing Fold 9/10
=====
```

Train size: 692, Test size: 76

[1/3] Training Random Forest...

Random Forest Accuracy: 0.6579

[2/3] Training SVM...

SVM Accuracy: 0.7237

[3/3] Training LSTM...

LSTM Accuracy: 0.6579

```
=====
Processing Fold 10/10
=====
```

Train size: 692, Test size: 76

[1/3] Training Random Forest...

Random Forest Accuracy: 0.8289

[2/3] Training SVM...

SVM Accuracy: 0.7763

[3/3] Training LSTM...

LSTM Accuracy: 0.7368

```
=====
All folds completed successfully!
=====
```

9. Display Results for Each Fold

9.1 Random Forest Results

```
In [39]: rf_df = pd.DataFrame(rf_results)

print("\n" + "="*80)
print("RANDOM FOREST - Per-Fold Results")
print("="*80)
print(rf_df.round(4).to_string(index=False))

rf_avg = rf_df.drop('Fold', axis=1).mean()
print("\n" + "="*80)
print("RANDOM FOREST - Average Results Across All Folds")
print("="*80)
for metric, value in rf_avg.items():
    print(f"{metric:20s}: {value:.4f}")
```

RANDOM FOREST - Per-Fold Results

	TP	TN	FP	FN	P	N	TPR	TNR	FPR	FNR	Precision	Recall	F1	Accuracy
y	Error_Rate	Balanced_Acc	TSS	HSS	AUC	Brier_Score	BSS	Fold						
16	37	13	11	27	50	0.5926	0.7400	0.2600	0.4074	0.5517	0.5926	0.5714	0.688	
3	0.3117	0.6663	0.3326	0.3270	0.7941	0.1796	0.2112	1						
20	41	8	8	28	49	0.7143	0.8367	0.1633	0.2857	0.7143	0.7143	0.7143	0.792	
2	0.2078	0.7755	0.5510	0.5510	0.8794	0.1401	0.3945	2						
15	42	10	10	25	52	0.6000	0.8077	0.1923	0.4000	0.6000	0.6000	0.6000	0.740	
3	0.2597	0.7038	0.4077	0.4077	0.7235	0.1989	0.0929	3						
15	48	7	7	22	55	0.6818	0.8727	0.1273	0.3182	0.6818	0.6818	0.6818	0.818	
2	0.1818	0.7773	0.5545	0.5545	0.9074	0.1166	0.4286	4						
19	44	5	9	28	49	0.6786	0.8980	0.1020	0.3214	0.7917	0.6786	0.7308	0.818	
2	0.1818	0.7883	0.5765	0.5947	0.8848	0.1370	0.4078	5						
14	39	5	19	33	44	0.4242	0.8864	0.1136	0.5758	0.7368	0.4242	0.5385	0.688	
3	0.3117	0.6553	0.3106	0.3280	0.7493	0.2100	0.1425	6						
20	42	8	7	27	50	0.7407	0.8400	0.1600	0.2593	0.7143	0.7407	0.7273	0.805	
2	0.1948	0.7904	0.5807	0.5758	0.8922	0.1245	0.4532	7						
8	48	9	12	20	57	0.4000	0.8421	0.1579	0.6000	0.4706	0.4000	0.4324	0.727	
3	0.2727	0.6211	0.2421	0.2545	0.7702	0.1702	0.1150	8						
15	35	10	16	31	45	0.4839	0.7778	0.2222	0.5161	0.6000	0.4839	0.5357	0.657	
9	0.3421	0.6308	0.2616	0.2698	0.7849	0.1908	0.2098	9						
17	46	3	10	27	49	0.6296	0.9388	0.0612	0.3704	0.8500	0.6296	0.7234	0.828	
9	0.1711	0.7842	0.5684	0.6035	0.8840	0.1338	0.4157	10						

RANDOM FOREST - Average Results Across All Folds

TP	: 15.9000
TN	: 42.2000
FP	: 7.8000
FN	: 10.9000
P	: 26.8000
N	: 50.0000
TPR	: 0.5946
TNR	: 0.8440
FPR	: 0.1560
FNR	: 0.4054
Precision	: 0.6711
Recall	: 0.5946
F1	: 0.6256
Accuracy	: 0.7565
Error_Rate	: 0.2435
Balanced_Acc	: 0.7193
TSS	: 0.4386
HSS	: 0.4467
AUC	: 0.8270
Brier_Score	: 0.1602
BSS	: 0.2871

9.2 SVM Results

```
In [40]: svm_df = pd.DataFrame(svm_results)
```

```
# Display results
print("\n" + "="*80)
print("SVM - Per-Fold Results")
print("="*80)
print(svm_df.round(4).to_string(index=False))

# Calculate and display averages
svm_avg = svm_df.drop('Fold', axis=1).mean()
print("\n" + "="*80)
print("SVM - Average Results Across All Folds")
print("="*80)
for metric, value in svm_avg.items():
    print(f"{metric:20s}: {value:.4f}")
```

```
=====
SVM - Per-Fold Results
=====
```

	TP	TN	FP	FN	P	N	TPR	TNR	FPR	FNR	Precision	Recall	F1	Accuracy
y	Error_Rate				Balanced_Acc		TSS	HSS	AUC	Brier_Score	BSS	Fold		
16	41	9	11	27	50	0.5926	0.8200	0.1800	0.4074	0.6400	0.5926	0.6154	0.740	
3		0.2597				0.7063	0.4126	0.4197	0.7800	0.1789	0.2143	1		
17	45	4	11	28	49	0.6071	0.9184	0.0816	0.3929	0.8095	0.6071	0.6939	0.805	
2		0.1948				0.7628	0.5255	0.5553	0.8258	0.1497	0.3533	2		
14	43	9	11	25	52	0.5600	0.8269	0.1731	0.4400	0.6087	0.5600	0.5833	0.740	
3		0.2597				0.6935	0.3869	0.3951	0.7531	0.1929	0.1202	3		
14	51	4	8	22	55	0.6364	0.9273	0.0727	0.3636	0.7778	0.6364	0.7000	0.844	
2		0.1558				0.7818	0.5636	0.5962	0.9017	0.1195	0.4143	4		
18	43	6	10	28	49	0.6429	0.8776	0.1224	0.3571	0.7500	0.6429	0.6923	0.792	
2		0.2078				0.7602	0.5204	0.5368	0.8812	0.1410	0.3907	5		
15	40	4	18	33	44	0.4545	0.9091	0.0909	0.5455	0.7895	0.4545	0.5769	0.714	
3		0.2857				0.6818	0.3636	0.3840	0.7775	0.1919	0.2162	6		
17	44	6	10	27	50	0.6296	0.8800	0.1200	0.3704	0.7391	0.6296	0.6800	0.792	
2		0.2078				0.7548	0.5096	0.5276	0.8785	0.1483	0.3487	7		
6	51	6	14	20	57	0.3000	0.8947	0.1053	0.7000	0.5000	0.3000	0.3750	0.740	
3		0.2597				0.5974	0.1947	0.2238	0.7868	0.1676	0.1281	8		
16	39	6	15	31	45	0.5161	0.8667	0.1333	0.4839	0.7273	0.5161	0.6038	0.723	
7		0.2763				0.6914	0.3828	0.4009	0.7885	0.1965	0.1864	9		
15	44	5	12	27	49	0.5556	0.8980	0.1020	0.4444	0.7500	0.5556	0.6383	0.776	
3		0.2237				0.7268	0.4535	0.4815	0.8632	0.1446	0.3685	10		

```
=====
SVM - Average Results Across All Folds
=====
```

```
TP : 14.8000
TN : 44.1000
FP : 5.9000
FN : 12.0000
P : 26.8000
N : 50.0000
TPR : 0.5495
TNR : 0.8819
FPR : 0.1181
FNR : 0.4505
Precision : 0.7092
Recall : 0.5495
F1 : 0.6159
Accuracy : 0.7669
Error_Rate : 0.2331
Balanced_Acc : 0.7157
TSS : 0.4313
HSS : 0.4521
AUC : 0.8236
Brier_Score : 0.1631
BSS : 0.2741
```

9.3 LSTM Results

```
In [41]: lstm_df = pd.DataFrame(lstm_results)
```

```
# Display results
print("\n" + "="*80)
print("LSTM - Per-Fold Results")
print("="*80)
print(lstm_df.round(4).to_string(index=False))

lstm_avg = lstm_df.drop('Fold', axis=1).mean()
print("\n" + "="*80)
print("LSTM - Average Results Across All Folds")
print("="*80)
for metric, value in lstm_avg.items():
    print(f"{metric:20s}: {value:.4f}")
```

```

=====
LSTM - Per-Fold Results
=====
  TP  TN  FP  FN  P  N    TPR    TNR    FPR    FNR  Precision  Recall  F1  Accurac
y  Error_Rate  Balanced_Acc  TSS    HSS    AUC  Brier_Score  BSS  Fold
18  35  15   9  27  50  0.6667  0.7000  0.3000  0.3333    0.5455  0.6667  0.6000    0.688
3    0.3117          0.6833  0.3667  0.3488  0.7533    0.1963  0.1379    1
22  37  12   6  28  49  0.7857  0.7551  0.2449  0.2143    0.6471  0.7857  0.7097    0.766
2    0.2338          0.7704  0.5408  0.5171  0.8703    0.1493  0.3550    2
12  44   8  13  25  52  0.4800  0.8462  0.1538  0.5200    0.6000  0.4800  0.5333    0.727
3    0.2727          0.6631  0.3262  0.3440  0.7300    0.2055  0.0629    3
15  46   9   7  22  55  0.6818  0.8364  0.1636  0.3182    0.6250  0.6818  0.6522    0.792
2    0.2078          0.7591  0.5182  0.5044  0.8455    0.1438  0.2956    4
18  44   5  10  28  49  0.6429  0.8980  0.1020  0.3571    0.7826  0.6429  0.7059    0.805
2    0.1948          0.7704  0.5408  0.5623  0.8491    0.1539  0.3350    5
18  34  10  15  33  44  0.5455  0.7727  0.2273  0.4545    0.6429  0.5455  0.5902    0.675
3    0.3247          0.6591  0.3182  0.3243  0.7514    0.2095  0.1447    6
19  44   6   8  27  50  0.7037  0.8800  0.1200  0.2963    0.7600  0.7037  0.7308    0.818
2    0.1818          0.7919  0.5837  0.5938  0.8896    0.1328  0.4169    7
 9  49   8  11  20  57  0.4500  0.8596  0.1404  0.5500    0.5294  0.4500  0.4865    0.753
2    0.2468          0.6548  0.3096  0.3255  0.7640    0.1660  0.1367    8
14  36   9  17  31  45  0.4516  0.8000  0.2000  0.5484    0.6087  0.4516  0.5185    0.657
9    0.3421          0.6258  0.2516  0.2621  0.7756    0.1925  0.2030    9
16  40   9  11  27  49  0.5926  0.8163  0.1837  0.4074    0.6400  0.5926  0.6154    0.736
8    0.2632          0.7045  0.4089  0.4158  0.8095    0.1666  0.2725   10

```

```

=====
LSTM - Average Results Across All Folds
=====
TP          : 16.1000
TN          : 40.9000
FP          : 9.1000
FN          : 10.7000
P           : 26.8000
N           : 50.0000
TPR         : 0.6000
TNR         : 0.8164
FPR         : 0.1836
FNR         : 0.4000
Precision   : 0.6381
Recall      : 0.6000
F1          : 0.6142
Accuracy    : 0.7421
Error_Rate  : 0.2579
Balanced_Acc : 0.7082
TSS         : 0.4165
HSS         : 0.4198
AUC         : 0.8038
Brier_Score : 0.1716
BSS         : 0.2360

```

10. Compare All Models - Summary Table

```

In [42]: comparison = pd.DataFrame({
          'Random_Forest': rf_avg,

```

```

        'SVM': svm_avg,
        'LSTM': lstm_avg
    })

    print("\n" + "="*90)
    print("MODEL COMPARISON - Average Performance Across 10 Folds")
    print("="*90)
    print(comparison.round(4))

    print("\n" + "="*90)
    print("Best Model for Each Metric:")
    print("="*90)
    for metric in comparison.index:
        best_model = comparison.loc[metric].idxmax()
        best_value = comparison.loc[metric].max()
        print(f"{metric:20s}: {best_model:15s} ({best_value:.4f})")

```

```
=====
=====
MODEL COMPARISON - Average Performance Across 10 Folds
=====
=====
```

	Random_Forest	SVM	LSTM
TP	15.9000	14.8000	16.1000
TN	42.2000	44.1000	40.9000
FP	7.8000	5.9000	9.1000
FN	10.9000	12.0000	10.7000
P	26.8000	26.8000	26.8000
N	50.0000	50.0000	50.0000
TPR	0.5946	0.5495	0.6000
TNR	0.8440	0.8819	0.8164
FPR	0.1560	0.1181	0.1836
FNR	0.4054	0.4505	0.4000
Precision	0.6711	0.7092	0.6381
Recall	0.5946	0.5495	0.6000
F1	0.6256	0.6159	0.6142
Accuracy	0.7565	0.7669	0.7421
Error_Rate	0.2435	0.2331	0.2579
Balanced_Acc	0.7193	0.7157	0.7082
TSS	0.4386	0.4313	0.4165
HSS	0.4467	0.4521	0.4198
AUC	0.8270	0.8236	0.8038
Brier_Score	0.1602	0.1631	0.1716
BSS	0.2871	0.2741	0.2360

```
=====
=====
Best Model for Each Metric:
=====
=====
```

TP	: LSTM	(16.1000)
TN	: SVM	(44.1000)
FP	: LSTM	(9.1000)
FN	: SVM	(12.0000)
P	: Random_Forest	(26.8000)
N	: Random_Forest	(50.0000)
TPR	: LSTM	(0.6000)
TNR	: SVM	(0.8819)
FPR	: LSTM	(0.1836)
FNR	: SVM	(0.4505)
Precision	: SVM	(0.7092)
Recall	: LSTM	(0.6000)
F1	: Random_Forest	(0.6256)
Accuracy	: SVM	(0.7669)
Error_Rate	: LSTM	(0.2579)
Balanced_Acc	: Random_Forest	(0.7193)
TSS	: Random_Forest	(0.4386)
HSS	: SVM	(0.4521)
AUC	: Random_Forest	(0.8270)
Brier_Score	: LSTM	(0.1716)
BSS	: Random_Forest	(0.2871)

11. Additional Model Performance Visualizations



Graph 6: Feature Importance (Random Forest)

```
In [43]: # Train final RF model for feature importance
rf_importance = RandomForestClassifier(n_estimators=100, random_state=42)
rf_importance.fit(x, y)

importance = rf_importance.feature_importances_
features = x.columns

indices = np.argsort(importance)[::-1]

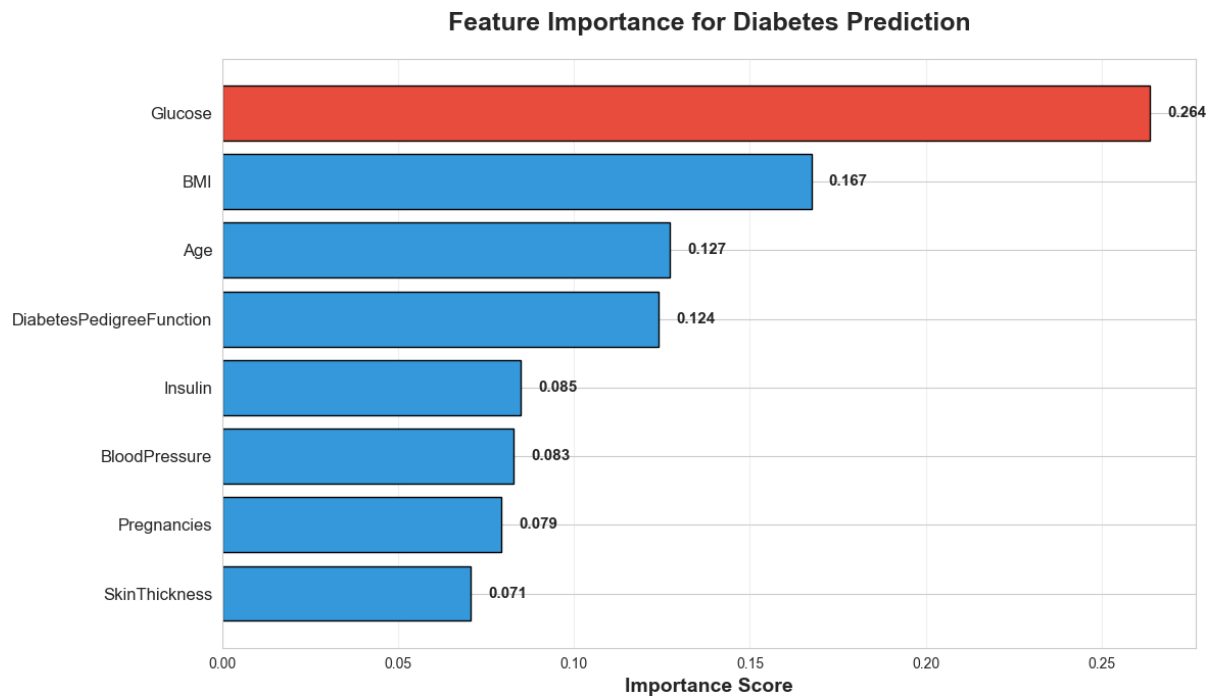
plt.figure(figsize=(12, 7))
colorsBars = ['#e74c3c' if i == 0 else '#3498db' for i in range(len(features))]
bars = plt.barh(range(len(features)), importance[indices],
                color=colorsBars, edgecolor='black')

plt.yticks(range(len(features)), [features[i] for i in indices], fontsize=12)
plt.xlabel('Importance Score', fontsize=14, weight='bold')
plt.title('Feature Importance for Diabetes Prediction',
          fontsize=18, weight='bold', pad=20)
plt.gca().invert_yaxis()
plt.grid(axis='x', alpha=0.3)

for i, bar in enumerate(bars):
    width = bar.get_width()
    plt.text(width + 0.005, bar.get_y() + bar.get_height()/2,
            f'{width:.3f}', ha='left', va='center',
            fontsize=11, weight='bold')

plt.tight_layout()
plt.savefig('graph6_feature_importance.png', dpi=300, bbox_inches='tight')
plt.show()

print("Graph 6 saved as 'graph6_feature_importance.png'")
```



Graph 6 saved as 'graph6_feature_importance.png'

Graph 7: Model Accuracy Comparison Bar Chart

```
In [44]: rf_accuracy = rf_avg['Accuracy']
svm_accuracy = svm_avg['Accuracy']
lstm_accuracy = lstm_avg['Accuracy']

plt.figure(figsize=(10, 7))
models = ['Random Forest', 'SVM', 'LSTM']
scores = [rf_accuracy, svm_accuracy, lstm_accuracy]
colors_models = ['#2ecc71', '#3498db', '#e74c3c']

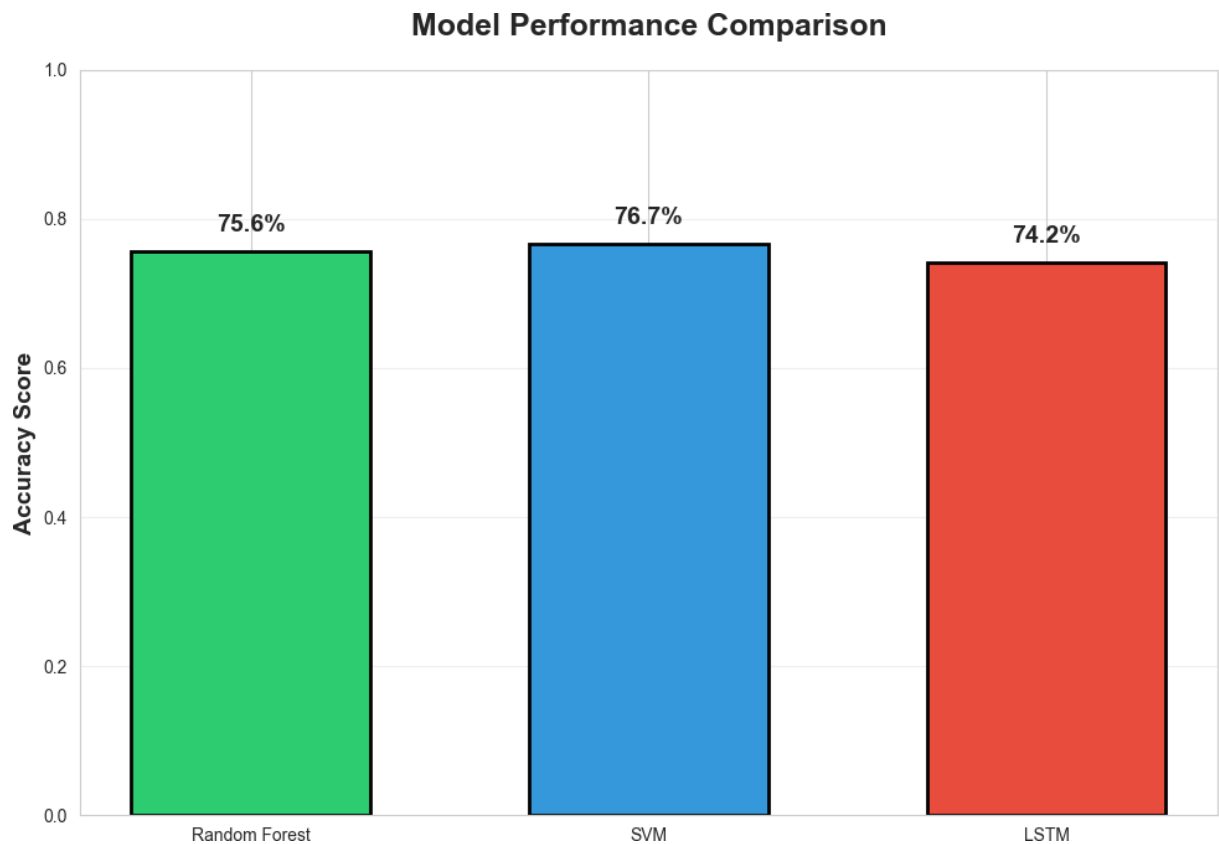
bars = plt.bar(models, scores, color=colors_models,
               edgecolor='black', linewidth=2, width=0.6)

plt.ylabel('Accuracy Score', fontsize=14, weight='bold')
plt.title('Model Performance Comparison', fontsize=18, weight='bold', pad=20)
plt.ylim([0, 1])
plt.grid(axis='y', alpha=0.3)

for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height + 0.02,
             f'{height:.1%}', ha='center', va='bottom',
             fontsize=14, weight='bold')

plt.tight_layout()
plt.savefig('graph7_model_comparison.png', dpi=300, bbox_inches='tight')
plt.show()

print("Graph 7 saved as 'graph7_model_comparison.png'")
```



Graph 7 saved as 'graph7_model_comparison.png'

Graph 8: ROC Curves for All Models

```
In [45]: x_train_viz, x_test_viz, y_train_viz, y_test_viz = train_test_split(
          x, y, test_size=0.2, random_state=42, stratify=y
        )

scaler_final = StandardScaler()
x_train_scaled_viz = scaler_final.fit_transform(x_train_viz)
x_test_scaled_viz = scaler_final.transform(x_test_viz)

# Random Forest
rf_final = RandomForestClassifier(n_estimators=100, random_state=42)
rf_final.fit(x_train_scaled_viz, y_train_viz)
rf_proba_viz = rf_final.predict_proba(x_test_scaled_viz)[:, 1]

# SVM
svm_final = SVC(kernel='rbf', probability=True, random_state=42)
svm_final.fit(x_train_scaled_viz, y_train_viz)
svm_proba_viz = svm_final.predict_proba(x_test_scaled_viz)[:, 1]

# LSTM
x_train_lstm_viz = x_train_scaled_viz.reshape((x_train_scaled_viz.shape[0], x_train_scaled_viz.shape[1], 1))
x_test_lstm_viz = x_test_scaled_viz.reshape((x_test_scaled_viz.shape[0], x_test_scaled_viz.shape[1], 1))

lstm_final = Sequential([
    LSTM(64, activation='relu', input_shape=(x_train_lstm_viz.shape[1], 1)),
    Dropout(0.2),
    Dense(32, activation='relu'),
```

```

        Dense(1, activation='sigmoid')
    ])
    lstm_final.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    lstm_final.fit(x_train_lstm_viz, y_train_viz, epochs=50, batch_size=32, verbose=0)
    lstm_proba_viz = lstm_final.predict(x_test_lstm_viz, verbose=0).flatten()

    fpr_rf, tpr_rf, _ = roc_curve(y_test_viz, rf_proba_viz)
    fpr_svm, tpr_svm, _ = roc_curve(y_test_viz, svm_proba_viz)
    fpr_lstm, tpr_lstm, _ = roc_curve(y_test_viz, lstm_proba_viz)

    auc_rf = auc(fpr_rf, tpr_rf)
    auc_svm = auc(fpr_svm, tpr_svm)
    auc_lstm = auc(fpr_lstm, tpr_lstm)

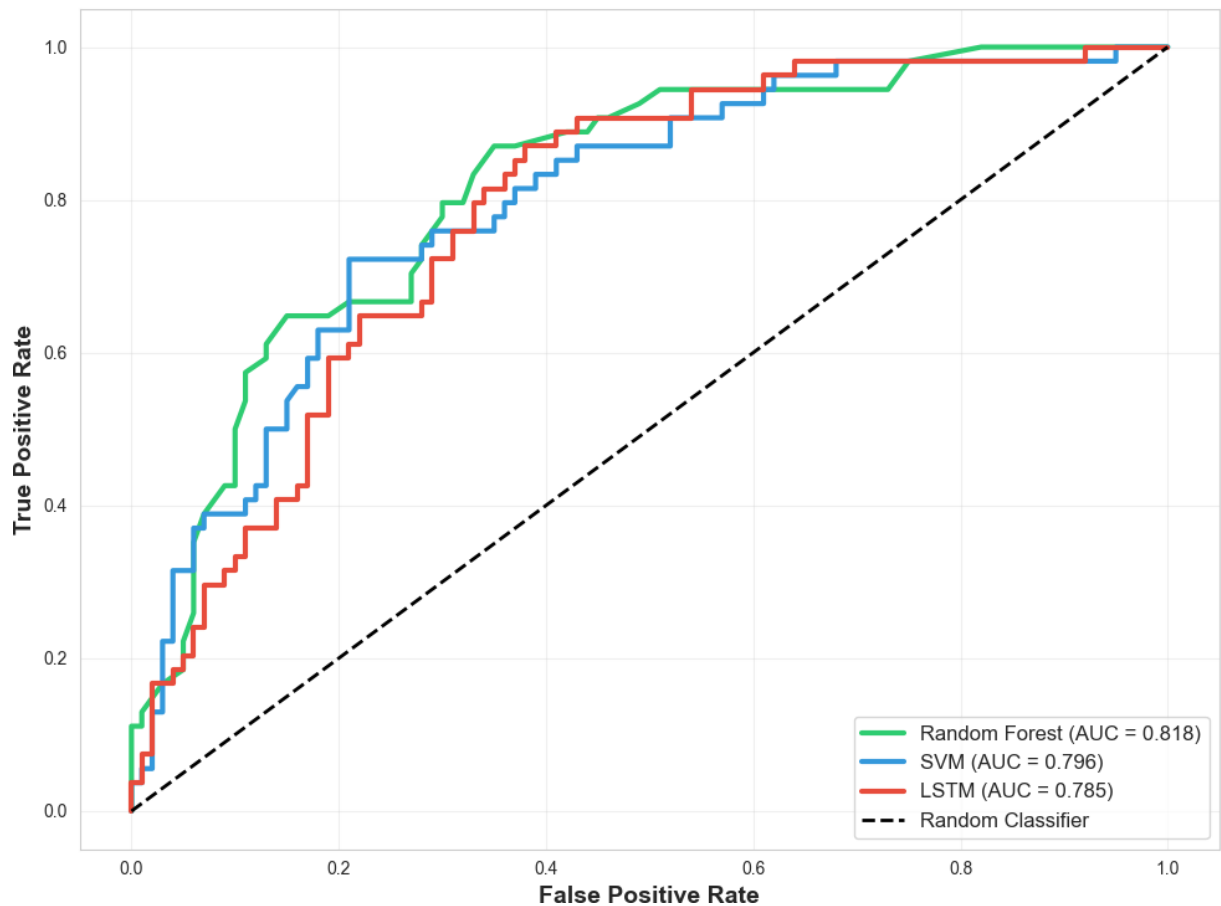
    plt.figure(figsize=(10, 8))
    plt.plot(fpr_rf, tpr_rf, label=f'Random Forest (AUC = {auc_rf:.3f})',
             linewidth=3, color='#2ecc71')
    plt.plot(fpr_svm, tpr_svm, label=f'SVM (AUC = {auc_svm:.3f})',
             linewidth=3, color='#3498db')
    plt.plot(fpr_lstm, tpr_lstm, label=f'LSTM (AUC = {auc_lstm:.3f})',
             linewidth=3, color='#e74c3c')
    plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier', linewidth=2)

    plt.xlabel('False Positive Rate', fontsize=14, weight='bold')
    plt.ylabel('True Positive Rate', fontsize=14, weight='bold')
    plt.title('ROC Curves - Model Comparison', fontsize=18, weight='bold', pad=20)
    plt.legend(fontsize=12, loc='lower right')
    plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.savefig('graph8_roc_curves.png', dpi=300, bbox_inches='tight')
    plt.show()

    print("\n📊 ROC-AUC Scores:")
    print(f"Random Forest: {auc_rf:.4f}")
    print(f"SVM: {auc_svm:.4f}")
    print(f"LSTM: {auc_lstm:.4f}")
    print("\nGraph 8 saved as 'graph8_roc_curves.png'")

```

ROC Curves - Model Comparison



ROC-AUC Scores:

Random Forest: 0.8179

SVM: 0.7964

LSTM: 0.7848

Graph 8 saved as 'graph8_roc_curves.png'

Graph 9: Confusion Matrix Heatmap (Random Forest)

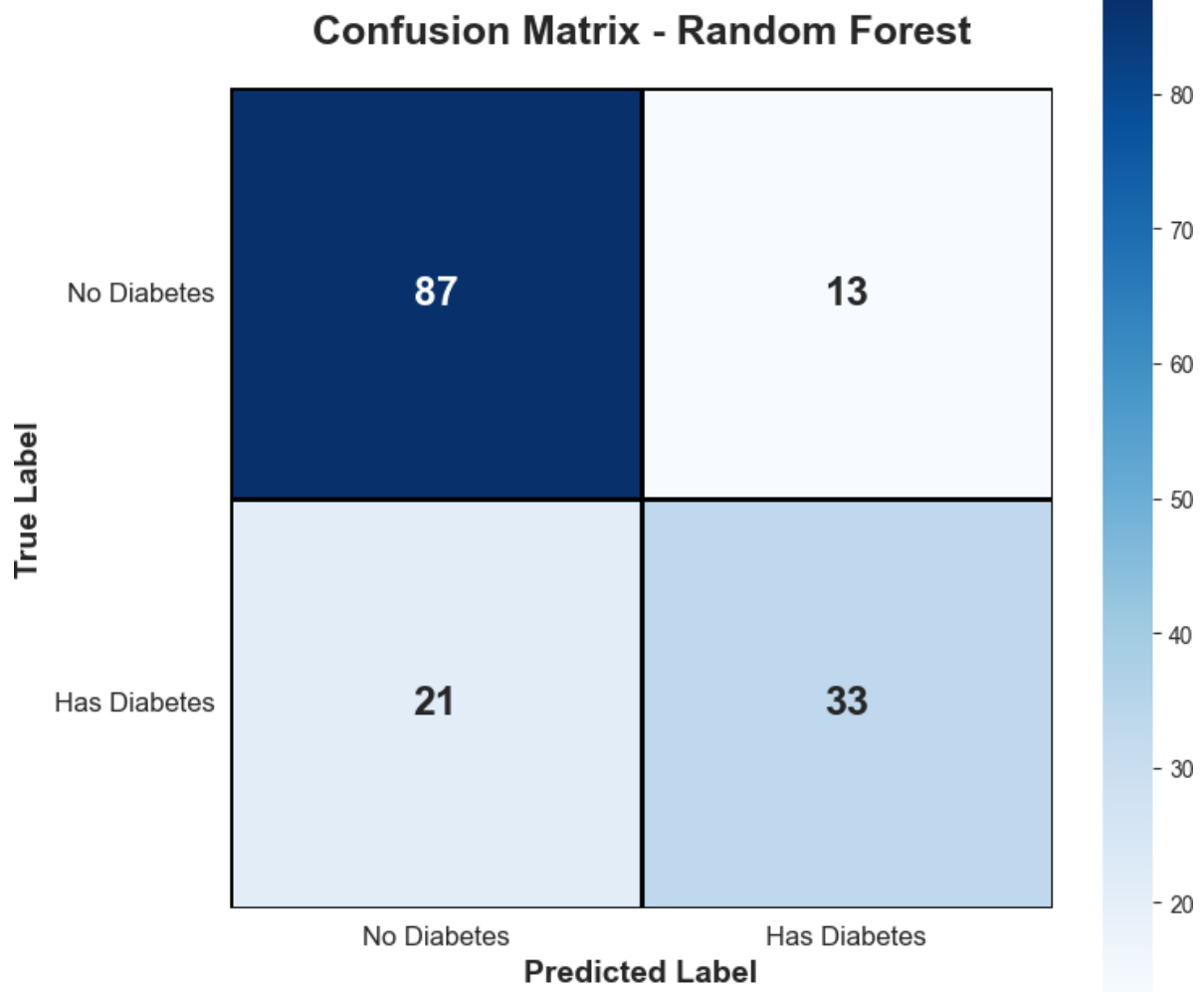
```
In [46]: rf_pred_viz = rf_final.predict(x_test_scaled_viz)
cm = confusion_matrix(y_test_viz, rf_pred_viz)

plt.figure(figsize=(8, 7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=True,
            square=True, linewidths=2, linecolor='black',
            annot_kws={'size': 18, 'weight': 'bold'})

plt.xlabel('Predicted Label', fontsize=14, weight='bold')
plt.ylabel('True Label', fontsize=14, weight='bold')
plt.title('Confusion Matrix - Random Forest', fontsize=18, weight='bold', pad=20)
plt.xticks([0.5, 1.5], ['No Diabetes', 'Has Diabetes'], fontsize=12)
plt.yticks([0.5, 1.5], ['No Diabetes', 'Has Diabetes'], fontsize=12, rotation=0)

plt.tight_layout()
plt.savefig('graph9_confusion_matrix.png', dpi=300, bbox_inches='tight')
plt.show()
```

```
print("Graph 9 saved as 'graph9_confusion_matrix.png'")
```



Graph 9 saved as 'graph9_confusion_matrix.png'

Graph 10: Accuracy Across Folds (Line Chart)

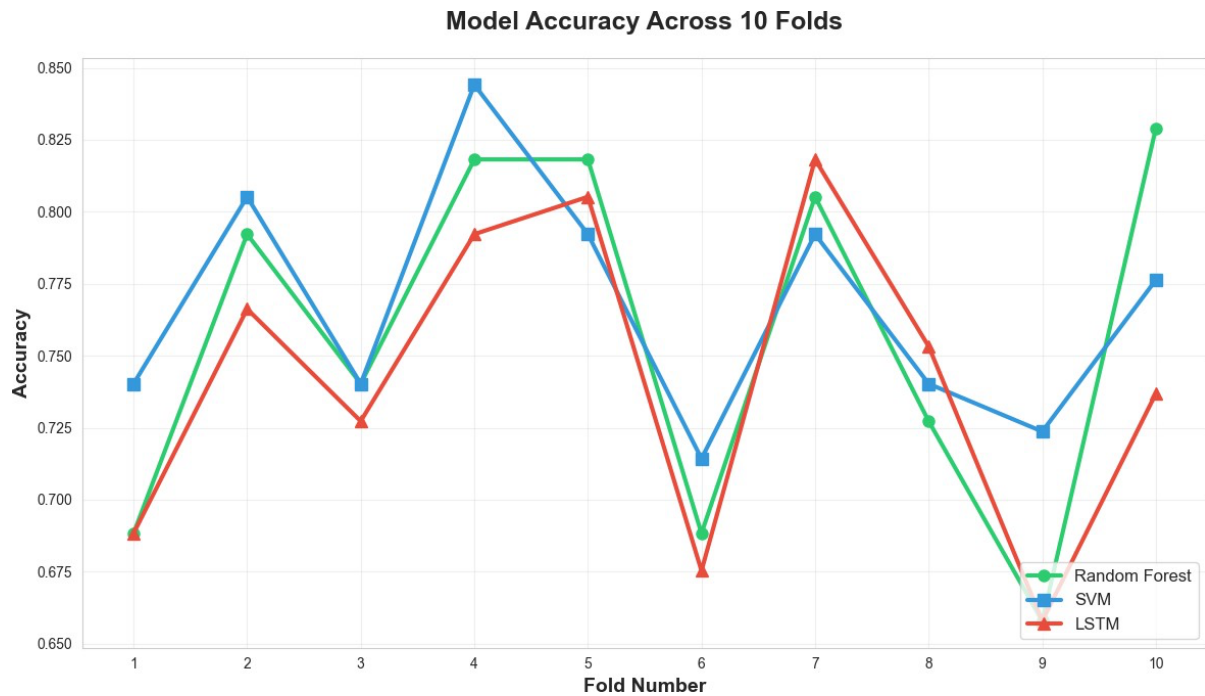
```
In [47]: plt.figure(figsize=(12, 7))
folds = list(range(1, num_folds + 1))

plt.plot(folds, rf_df['Accuracy'], marker='o', label='Random Forest',
         linewidth=3, markersize=8, color='#2ecc71')
plt.plot(folds, svm_df['Accuracy'], marker='s', label='SVM',
         linewidth=3, markersize=8, color='#3498db')
plt.plot(folds, lstm_df['Accuracy'], marker='^', label='LSTM',
         linewidth=3, markersize=8, color='#e74c3c')

plt.xlabel('Fold Number', fontsize=14, weight='bold')
plt.ylabel('Accuracy', fontsize=14, weight='bold')
plt.title('Model Accuracy Across 10 Folds', fontsize=18, weight='bold', pad=20)
plt.legend(fontsize=12, loc='lower right')
plt.grid(True, alpha=0.3)
plt.xticks(folds)
plt.tight_layout()
plt.savefig('graph10_accuracy_folds.png', dpi=300, bbox_inches='tight')
```

```
plt.show()

print("Graph 10 saved as 'graph10_accuracy_folds.png'")
```



Graph 10 saved as 'graph10_accuracy_folds.png'

Graph 11: Key Metrics Comparison (5 Subplots)

```
In [48]: metrics_to_plot = ['Accuracy', 'Precision', 'Recall', 'F1', 'AUC']

fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()

for idx, metric in enumerate(metrics_to_plot):
    ax = axes[idx]
    models = ['Random Forest', 'SVM', 'LSTM']
    values = [rf_avg[metric], svm_avg[metric], lstm_avg[metric]]

    bars = ax.bar(models, values, color=['#2ecc71', '#3498db', '#e74c3c'],
                  edgecolor='black', linewidth=1.5)
    ax.set_ylabel(metric, fontsize=12, weight='bold')
    ax.set_title(f'{metric} Comparison', fontsize=13, fontweight='bold')
    ax.set_ylim([0, 1])
    ax.grid(axis='y', alpha=0.3)

    for bar in bars:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height + 0.02,
                f'{height:.3f}', ha='center', va='bottom',
                fontsize=11, weight='bold')

fig.delaxes(axes[5])

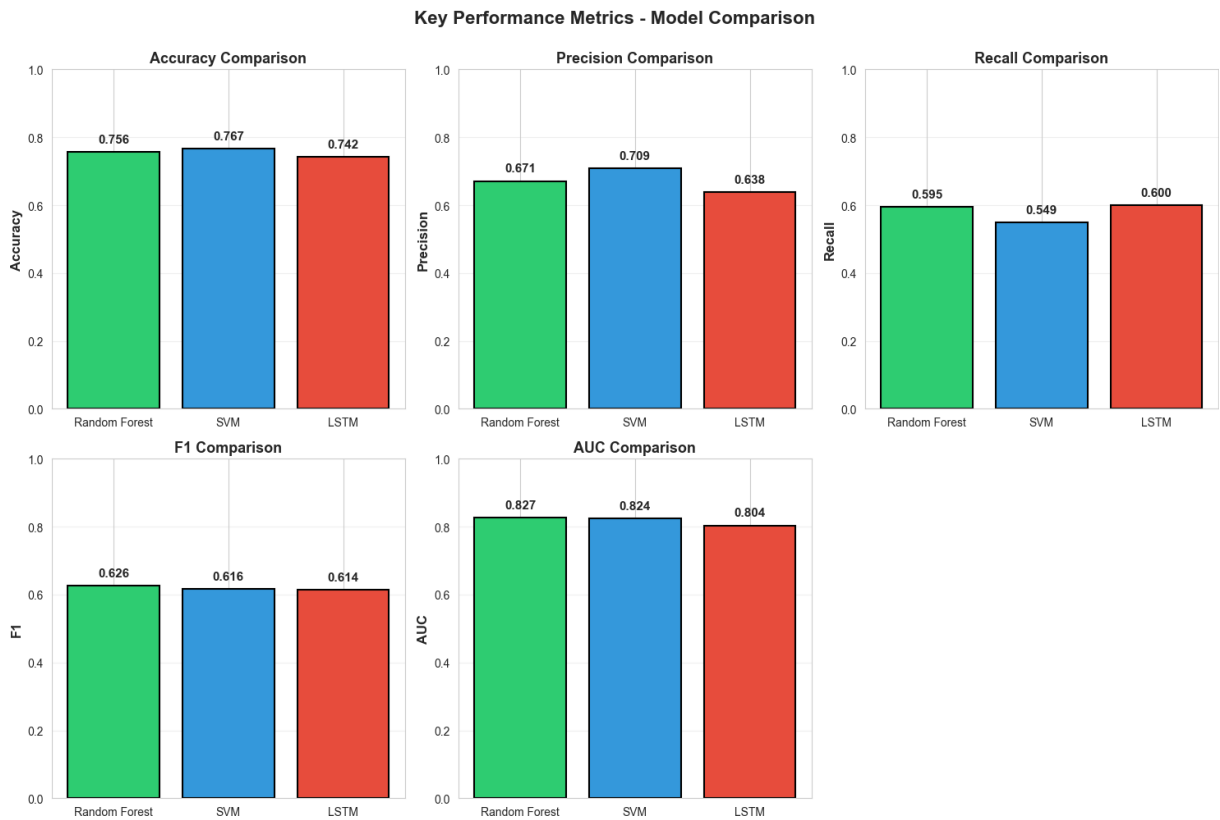
plt.suptitle('Key Performance Metrics - Model Comparison',
```

```

        fontsize=16, weight='bold', y=1.00)
plt.tight_layout()
plt.savefig('graph11_metrics_comparison.png', dpi=300, bbox_inches='tight')
plt.show()

print("Graph 11 saved as 'graph11_metrics_comparison.png'")

```



Graph 11 saved as 'graph11_metrics_comparison.png'



Graph 12: Individual ROC Curve - Random Forest

```

In [49]: # Individual ROC curve for Random Forest
plt.figure(figsize=(10, 8))

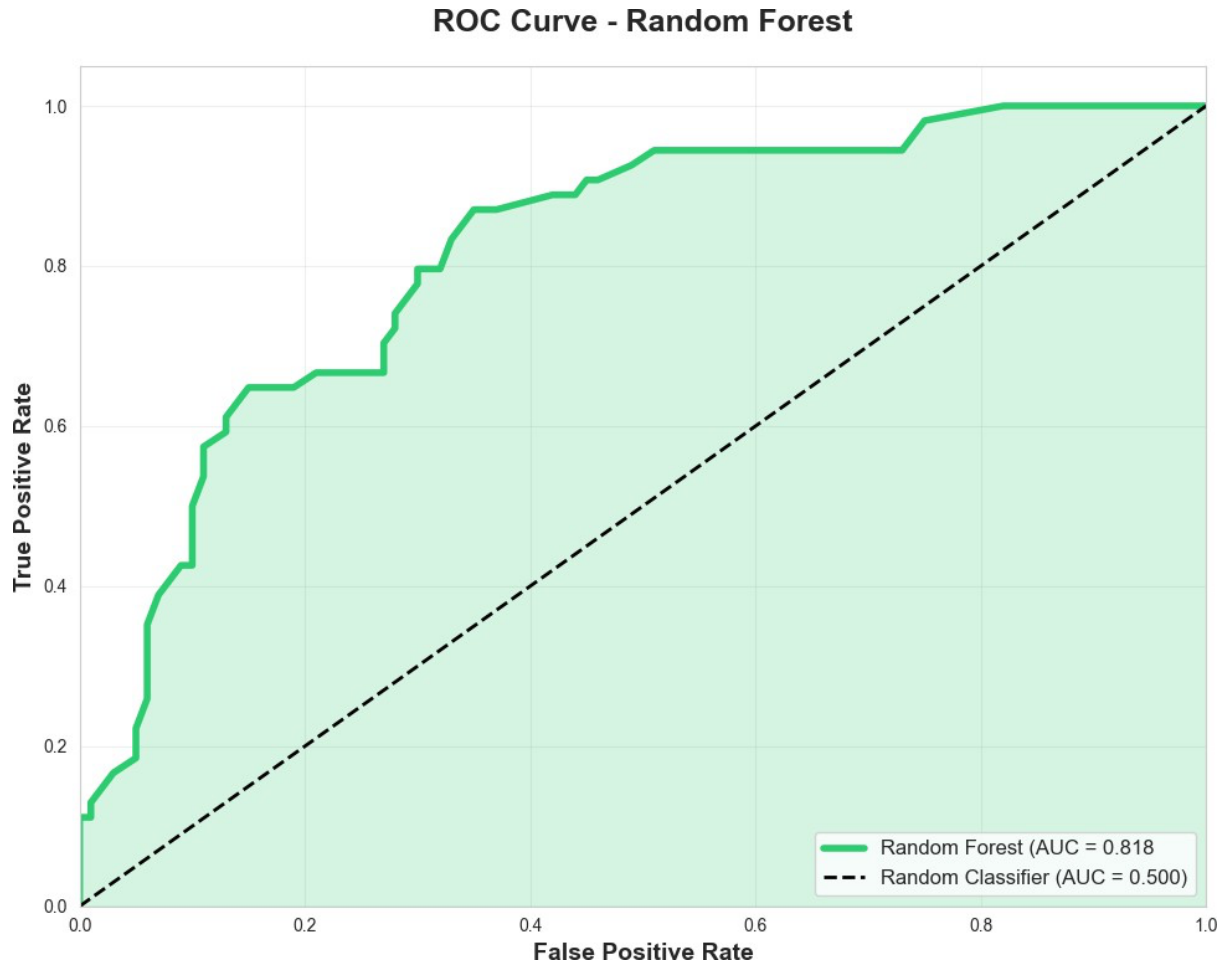
plt.plot(fpr_rf, tpr_rf, color='#2ecc71', linewidth=4,
        label=f'Random Forest (AUC = {auc_rf:.3f})')
plt.plot([0, 1], [0, 1], 'k--', linewidth=2, label='Random Classifier (AUC = 0.500)')

plt.fill_between(fpr_rf, tpr_rf, alpha=0.2, color='#2ecc71')

plt.xlabel('False Positive Rate', fontsize=14, weight='bold')
plt.ylabel('True Positive Rate', fontsize=14, weight='bold')
plt.title('ROC Curve - Random Forest', fontsize=18, weight='bold', pad=20)
plt.legend(fontsize=12, loc='lower right')
plt.grid(alpha=0.3)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.tight_layout()
plt.savefig('graph12_roc_random_forest.png', dpi=300, bbox_inches='tight')
plt.show()

```

```
print(f"Graph 12 saved as 'graph12_roc_random_forest.png'")
print(f"    Random Forest AUC: {auc_rf:.4f}")
```



Graph 12 saved as 'graph12_roc_random_forest.png'
Random Forest AUC: 0.8179

Graph 13: Individual ROC Curve - SVM

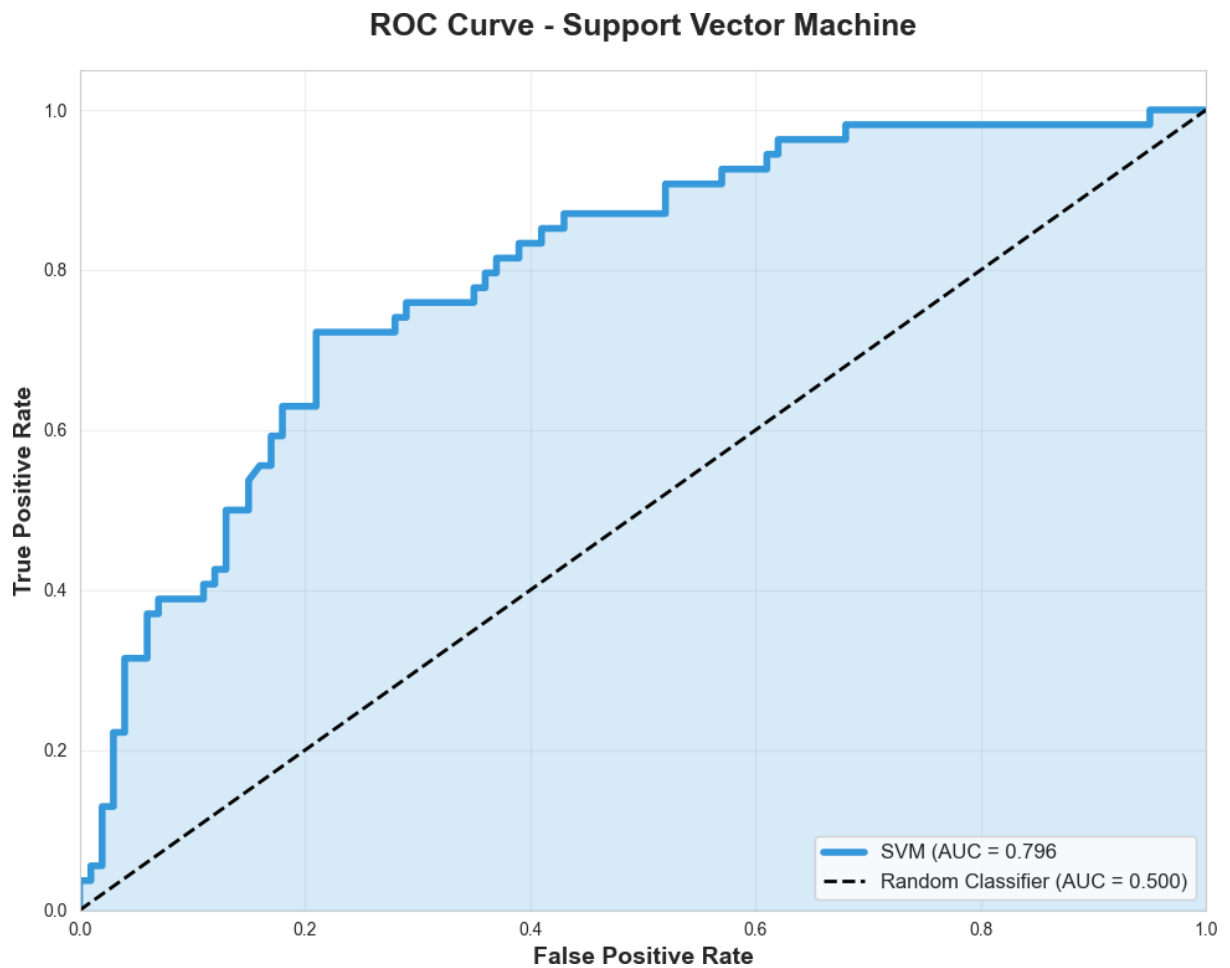
```
In [50]: # Individual ROC curve for SVM
plt.figure(figsize=(10, 8))

plt.plot(fpr_svm, tpr_svm, color='#3498db', linewidth=4,
         label=f'SVM (AUC = {auc_svm:.3f})')
plt.plot([0, 1], [0, 1], 'k--', linewidth=2, label='Random Classifier (AUC = 0.500)')

plt.fill_between(fpr_svm, tpr_svm, alpha=0.2, color='#3498db')

plt.xlabel('False Positive Rate', fontsize=14, weight='bold')
plt.ylabel('True Positive Rate', fontsize=14, weight='bold')
plt.title('ROC Curve - Support Vector Machine', fontsize=18, weight='bold', pad=20)
plt.legend(fontsize=12, loc='lower right')
plt.grid(alpha=0.3)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.tight_layout()
plt.savefig('graph13_roc_svm.png', dpi=300, bbox_inches='tight')
plt.show()
```

```
print(f"Graph 13 saved as 'graph13_roc_svm.png'")
print(f"    SVM AUC: {auc_svm:.4f}")
```



Graph 13 saved as 'graph13_roc_svm.png'
SVM AUC: 0.7964

Graph 14: Individual ROC Curve - LSTM

```
In [51]: # Individual ROC curve for LSTM
plt.figure(figsize=(10, 8))

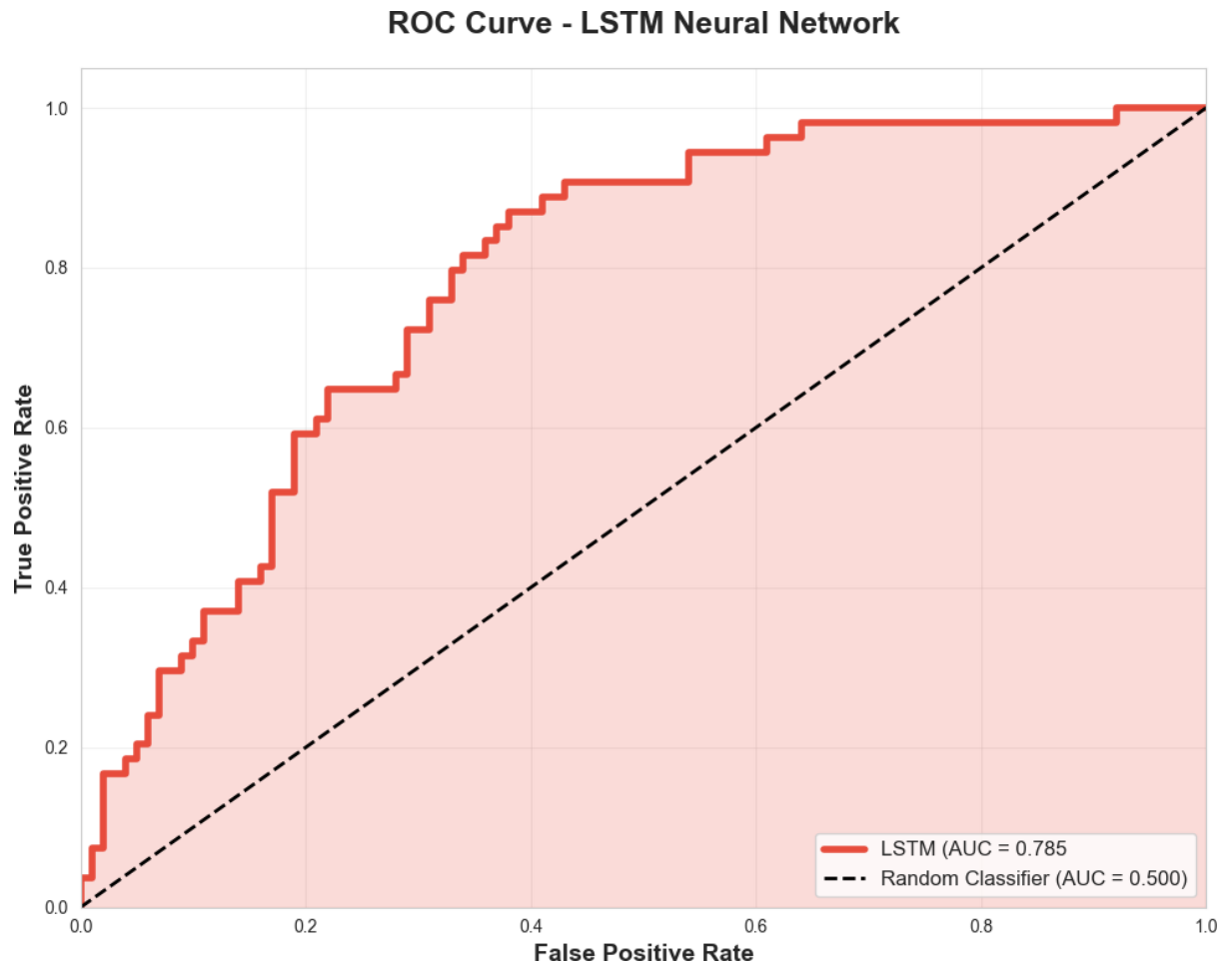
plt.plot(fpr_lstm, tpr_lstm, color='#e74c3c', linewidth=4,
         label=f'LSTM (AUC = {auc_lstm:.3f})')
plt.plot([0, 1], [0, 1], 'k--', linewidth=2, label='Random Classifier (AUC = 0.500)')

plt.fill_between(fpr_lstm, tpr_lstm, alpha=0.2, color='#e74c3c')

plt.xlabel('False Positive Rate', fontsize=14, weight='bold')
plt.ylabel('True Positive Rate', fontsize=14, weight='bold')
plt.title('ROC Curve - LSTM Neural Network', fontsize=18, weight='bold', pad=20)
plt.legend(fontsize=12, loc='lower right')
plt.grid(alpha=0.3)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.tight_layout()
plt.savefig('graph14_roc_lstm.png', dpi=300, bbox_inches='tight')
```

```
plt.show()

print(f"Graph 14 saved as 'graph14_roc_lstm.png'")
print(f"    LSTM AUC: {auc_lstm:.4f}")
```



Graph 14 saved as 'graph14_roc_lstm.png'
LSTM AUC: 0.7848

```
In [52]: # Save individual fold results
rf_df.to_csv('random_forest_results.csv', index=False)
svm_df.to_csv('svm_results.csv', index=False)
lstm_df.to_csv('lstm_results.csv', index=False)

comparison.to_csv('model_comparison_summary.csv')

print("Results exported successfully!")
print("\nFiles created:")
print("random_forest_results.csv")
print("svm_results.csv")
print("lstm_results.csv")
print("model_comparison_summary.csv")
```

Results exported successfully!

Files created:

random_forest_results.csv

svm_results.csv

lstm_results.csv

model_comparison_summary.csv